

4.1.5 การเรียงแบบเลือก (Selection Sort)

เป็นวิธีเรียงข้อมูลที่ง่ายที่สุด และถือว่าเป็นพื้นฐานของการเรียงข้อมูลตัวอื่น ๆ หลักการมีอยู่ว่าจะเริ่มต้นค้นหาข้อมูลตัวที่น้อยที่สุดในชุดข้อมูลที่เข้ามา (กรณีเรียงจากน้อยไปมาก) หลังจากนั้นเอาข้อมูลตัวที่น้อยที่สุดมาเก็บไว้เป็นตัวแรก แล้วกลับไปหาข้อมูลตัวที่น้อยที่สุดในชุดข้อมูลที่เหลืมาเรียงต่อจากตัวแรกไปจนกว่าจะหมด ในที่สุดก็จะได้ข้อมูลที่เรียงจากน้อยไปมาก สามารถจะสรุปหลักการของการเรียงแบบเลือกได้ดังนี้

เมื่อมีชุดข้อมูลเข้ามา จะเริ่มทำการเรียงโดย

1. ค้นหาข้อมูลค่าที่น้อยที่สุดจากข้อมูลทั้งหมด แล้วนำมาแลกเปลี่ยนกับตำแหน่งแรก
2. ค้นหาค่าที่น้อยที่สุดต่อไปจากชุดข้อมูลที่เหลือ แล้วนำมาวางไว้ในลำดับถัดจากตำแหน่งแรก
3. ทำข้อ 2 ซ้ำไปเรื่อย ๆ จนถึงค่าสุดท้ายหรือจนกว่าข้อมูลจะหมดจากชุดข้อมูลนั้น ๆ

สมมติมีข้อมูลอยู่ดังนี้ 77, 33, 44, 11, 88, 22, 66, 55 สามารถจัดเรียงได้ดังนี้

ตำแหน่งที่ น้อยที่สุด	A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]
LOC = 4	<u>77</u>	33	44	<u>11</u>	88	22	66	55
LOC = 6	11	<u>33</u>	44	77	88	<u>22</u>	66	55
LOC = 6	11	22	<u>44</u>	77	88	<u>33</u>	66	55
LOC = 6	11	22	33	<u>77</u>	88	<u>44</u>	66	55
LOC = 8	11	22	33	44	<u>88</u>	77	66	<u>55</u>
LOC = 7	11	22	33	44	55	<u>77</u>	<u>66</u>	88
LOC = 7	11	22	33	44	55	66	<u>77</u>	88
Sorted	11	22	33	44	55	66	77	88

รูปที่ 4.11 แสดงการเรียงข้อมูลแบบเลือก

การวางโครงสร้างโปรแกรมการเรียงแบบเลือก

ลักษณะการเรียงแบบเลือกมี 2 ขั้นตอนใหญ่ ๆ คือ

1. เลือกค่าต่ำสุดจากลิสต์ (กรณีเรียงจากน้อยไปมาก)
2. นำค่าที่เลือกใน (1) สลับค่ากับตำแหน่งแรกของลิสต์ ข้อมูลในลิสต์ที่ยังไม่ได้เรียงจะลดลง 1 กลับไปทำขั้นตอน (1) จนกว่าจะหมดข้อมูล

ดังนั้นจึงแบ่งเป็น 2 โมดูลดังนี้

1. โมดูลหาค่าต่ำสุดจากลิสต์ (Minimum) : มี j เป็นตัวแปรรับค่าตำแหน่งต้น ลิสต์ในอาร์เรย์ และจะคืนค่าเป็นตำแหน่งของค่าที่มีค่าต่ำสุดในลิสต์

2. **โมดูลเลือกสลับค่า (SelectionSort)** : มี N เป็นตัวแปรรับค่าตำแหน่งสูงสุด ของอาร์เรย์ โดยใช้คำสั่ง for ในการวนรอบตั้งแต่ 1..N โดยต้นลิสต์จะเริ่มต้นที่ตำแหน่ง 1 แล้ว ส่งผ่านข้อมูลทั้งหมดไปให้ฟังก์ชัน **Minimum** เพื่อหาตำแหน่งของค่าต่ำสุด แล้วทำการสลับ กับค่าของตำแหน่งที่ 1 จากนั้นต้นลิสต์จะเริ่มต้นที่ 2 แล้วส่งผ่านข้อมูลตั้งแต่ 2..N ไปให้ ฟังก์ชัน **Minimum** เพื่อหาตำแหน่งของค่าต่ำสุด แล้วทำการสลับกับค่าของตำแหน่งที่ 2 ทำ อย่างนี้ไปเรื่อย ๆ จนกว่าจะหมดชุดข้อมูลก็จะได้ข้อมูลที่เรียงกันแล้วเสร็จ

4.1.5.1 โปรแกรมการเรียงแบบเลือก

```
/* Program ASCENDING SELECTION SORT can..
1. Random raw data into 1 dimension Array
2. Sorting and display detail of each result
3. Display final result
*/
#include <stdio.h> //use printf
#include <conio.h> //use getch
#include <stdlib.h> //use random
#include <time.h> //use time
#define MaxData 100 // Define Max Data
int Data[MaxData];
int N;
void PrepareRawData(int N)
{
    int i;
    srand(time(NULL)); //for difference random number in rand()
    for (i=1;i<=N;i++)
        Data[i]=1+rand() % 99; //random difference number 1..99
}
void DispData(int N,int out) //Show Data in array 2 format
{
    int i;
    for(i=1;i<=N;i++)
    {
        if(out>=i)
            printf("[%2d] ",Data[i]); //show [] if it's Output
        else
            printf("%2d  ",Data[i]); //show normal form it is not Output
    }
    printf("\n");
}
void swap(int a,int b)
{
    int temp;
```

```

    temp=Data[a];
    Data[a]=Data[b];
    Data[b]=temp;
}
int Minimum(int j) //Find Minimum data in Data[] between j..N
{
    int i,temp,Location;
    Location=j; //set first Location
    temp=Data[j]; //set begin value
    for(i=j+1;i<=N;i++)
    {
        if(temp>Data[i])
        {
            temp=Data[i]; //change new minimum data
            Location=i; //keep new Location
        }
    }
    return(Location); //return address of Minimum data
} //End Fn.

void SelectionSort(int N)
{
    int i,j,Location;
    printf("-----\n");
    printf(" i LOC ");
    for(i=1;i<=N;i++)
        printf("(%2d) ",i);
    printf("\n");
    DispData(N,0); //Show every step sorting
    printf("-----\n");
    for(i=1;i<=N;i++)
    {
        Location=Minimum(i); //Find Minimum Location between i..N
        swap(i,Location);
        printf("(%2d) (%2d) ",i,Location); //show location of array
        DispData(N,i); //Show every step sorting
    }
} //End Fn.

int main()
{
    printf("ASCENDING SELECTION SORT\n");
    printf("=====\n");

```

```

N=12; //Change amount of N here
PrepareRawData(N);
printf("Raw Data...");
DispData(N,0);
printf("Processing Data...\n");
SelectionSort(N);
printf("-----\n");
printf("Sorted Data : ");
DispData(N,N); //Sorted Data
getch();
return(0);
} //End Main

```

```

C:\Users\LENOVO\Downloads\Code\Code DataStructure65\24. Selection Sort\Selection Sort.exe
ASCENDING SELECTION SORT
=====
Raw Data... 9 79 75 13 97 93 90 92 20 40 11 48
Processing Data...
=====
i LOC ( 1) ( 2) ( 3) ( 4) ( 5) ( 6) ( 7) ( 8) ( 9) (10) (11) (12)
      9 79 75 13 97 93 90 92 20 40 11 48
=====
( 1) ( 1) [ 9] 79 75 13 97 93 90 92 20 40 11 48
( 2) (11) [ 9] [11] 75 13 97 93 90 92 20 40 79 48
( 3) ( 4) [ 9] [11] [13] 75 97 93 90 92 20 40 79 48
( 4) ( 9) [ 9] [11] [13] [20] 97 93 90 92 75 40 79 48
( 5) (10) [ 9] [11] [13] [20] [40] 93 90 92 75 97 79 48
( 6) (12) [ 9] [11] [13] [20] [40] [48] 90 92 75 97 79 93
( 7) ( 9) [ 9] [11] [13] [20] [40] [48] [75] 92 90 97 79 93
( 8) (11) [ 9] [11] [13] [20] [40] [48] [75] [79] 90 97 92 93
( 9) ( 9) [ 9] [11] [13] [20] [40] [48] [75] [79] [90] 97 92 93
(10) (11) [ 9] [11] [13] [20] [40] [48] [75] [79] [90] [92] 97 93
(11) (12) [ 9] [11] [13] [20] [40] [48] [75] [79] [90] [92] [93] 97
(12) (12) [ 9] [11] [13] [20] [40] [48] [75] [79] [90] [92] [93] [97]
=====
Sorted Data : [ 9] [11] [13] [20] [40] [48] [75] [79] [90] [92] [93] [97]

```

รูปที่ 4.12 แสดงการทำงานของโปรแกรมเรียงแบบเลือก

4.1.6 การเรียงแบบเรดิค (Radix Sort)

การเรียงแบบนี้ จะใช้แต่ละตำแหน่งตัวเลขของแต่ละคีย์ โดยเริ่มตั้งแต่ตำแหน่ง นัยสำคัญน้อยที่สุด (Least Significant Digit) การเรียงจะทำการเตรียมถึง 10 ถัง (กรณีเป็น เลขฐานสิบ) แล้วในแต่ละเที่ยวของการเรียงจะนำคีย์ใส่ไปเข้าแต่ละถังตามค่าตัวเลขในแต่ละตำแหน่ง

มีชุดข้อมูลดังนี้ 156, 425, 679, 128, 420, 067, 772, 895, 835

การเรียงเที่ยวที่ 1 กระจายคีย์ไปตามถังต่าง ๆ ตามค่าตัวเลขที่ตำแหน่งขวาสุด (หลักหน่วย)

	<u>156</u>	<u>425</u>	<u>679</u>	<u>128</u>	<u>420</u>	<u>067</u>	<u>772</u>	<u>895</u>	<u>835</u>
ถึงที่ 0	420						ถึงที่ 5	425	895 835
ถึงที่ 1							ถึงที่ 6	156	
ถึงที่ 2	772						ถึงที่ 7	067	
ถึงที่ 3							ถึงที่ 8	128	
ถึงที่ 4							ถึงที่ 9	679	

เมื่ออ่านค่าในแต่ละถึงออกมาตามลำดับจะได้ค่าต่าง ๆ ดังนี้

420, 772, 425, 895, 835, 156, 067, 128, 679

การเรียงเที่ยวที่ 2 ในเที่ยวนี้ จะพิจารณาหลักกลาง (หลักสิบ)

	<u>420</u>	<u>772</u>	<u>425</u>	<u>895</u>	<u>835</u>	<u>156</u>	<u>067</u>	<u>128</u>	<u>679</u>
ถึงที่ 0							ถึงที่ 5	156	
ถึงที่ 1							ถึงที่ 6	067	
ถึงที่ 2	420	425	128				ถึงที่ 7	722	679
ถึงที่ 3	835						ถึงที่ 8		
ถึงที่ 4							ถึงที่ 9	895	

เมื่ออ่านค่าต่าง ๆ ในแต่ละถึงออกมาตามลำดับจะได้

420, 425, 128, 835, 156, 067, 772, 679, 895

การเรียงเที่ยวที่ 3 ในเที่ยวนี้ จะพิจารณาดำแหน่งซ้ายสุด (หลักร้อย)

	<u>420</u>	<u>425</u>	<u>128</u>	<u>835</u>	<u>156</u>	<u>067</u>	<u>772</u>	<u>679</u>	<u>895</u>
ถึงที่ 0	067						ถึงที่ 5		
ถึงที่ 1	128	156					ถึงที่ 6	679	
ถึงที่ 2							ถึงที่ 7	722	
ถึงที่ 3							ถึงที่ 8	853	895
ถึงที่ 4	420	425					ถึงที่ 9		

เมื่ออ่านค่าต่าง ๆ ในแต่ละถึงออกมาจะได้ชุดคี่ที่เรียงแล้วเป็นดังนี้

067, 128, 156, 420, 425, 679, 772, 835, 895

การวางโครงสร้างโปรแกรมการเรียงแบบเรดิก

เตรียมโครงสร้างอาร์เรย์ 2 มิติ Radix[0][20] เพื่อเป็นถังสำหรับโยนค่าแต่ละหลัก โดยขั้นตอนจะใช้คำสั่ง itoa แปลงตัวเลขในอาร์เรย์แต่ละตัวของตัวแปร Data ไปเป็นข้อความ แล้วเก็บไว้ในตัวแปร txt จากข้อความในตัวแปร txt เริ่มแรกจะนำข้อความเฉพาะหลักหน่วย (ขวาสุด) มาแปลงเป็นตัวเลขเพื่อใช้เป็นคีย์ในการโยนค่าข้อมูลในอาร์เรย์ Data ลงไปในแต่ละ แถวของถัง Radix ซึ่งการโยนจะเป็นแบบสแตก (Stack) นำข้อมูลในตัวแปร Data มาโยนทีละตัว ๆ จนกว่าจะครบทุกตัว

อ่านข้อมูลในถัง Radix ตั้งแต่ถังที่ 0..9 ออกมาเก็บไว้ในตัวแปร Data การอ่านจะเป็นแบบคิว (Queue) และทำการล้าง (Clear) ข้อมูลในถังให้เป็นค่าว่าง

ทำการโยนข้อมูลของตัวแปร Data ลงถังอีกครั้ง ตอนนี้พิจารณาโยนหลักร้อยลงถัง Radix โดยทำการโยนจนครบทุกตัว และอ่านออกมาเก็บในตัวแปร Data แล้วทำการล้างถัง

โยนข้อมูล Data หลักร้อยลงถัง Radix จนครบ อ่านข้อมูลออกมา สุดท้ายจะได้ข้อมูลที่เรียงกันจากน้อยไปมาก

4.1.6.1 โปรแกรมการเรียงแบบเรดิก

```
/* Program ASCENDING RADIX SORT can..
1. Random raw data into 1 dimension Array
2. Sorting and display detail of each step result
3. Display final result
*/
#include <stdio.h> //use printf
#include <conio.h> //use getch
#include <stdlib.h> //use random
#include <time.h> //use time
#define MaxData 100 // Define Max Data
#define MaxRow 10 //0..9 in Decimal base
#define MaxCol 20 //0..19
int Data[MaxData];
int Radix[MaxRow][MaxCol]; //Radix is temporary tank. The size is
[0..MaxRow,0..MaxCol]
int N,N1;
void ClearStackPT() //Clear every block=0 and use Radix[0] for SP
{
    int i;
    for(i=0;i<=MaxRow;i++)
        Radix[i][0]=NULL; //This position keep the SP
}
void PrepareRawData(int N2)
{
    int i;
```

```

    srand(time(NULL)); //for difference random number in rand()
    for (i=0;i<N2;i++)
        Data[i]=(rand() % 899)+100; //random difference number 100..999
}
void DispData(int N2) //Show Data in array
{
    int i;
    for(i=0;i<N2;i++)
        printf("%3d ",Data[i]);
    printf("\n");
}
void Push(int Rad, int Dat) //Put data into Parallel Stack by keep SP
at (Rad,0)
{
    int SP;
    SP=Radix[Rad][0]+1; //Skip SP
    Radix[Rad][0]=SP; //keep new SP
    Radix[Rad][SP]=Dat; //Push data into that Radix
}
void ReadStack() //Read data from each Stack and transfer to Data[]
{
    int i,j,k,SP;
    k=0; //Start of Data[] at subscript 0
    for(i=0;i<=MaxRow;i++)
    {
        SP=Radix[i][0]; //Max Count value of each Radix keep at column 0
        for(j=1;j<=SP;j++)
        {
            Data[k]=Radix[i][j]; //Transfer data from every Stack to Data[]
            k++;
        }
    } //End for
} //End Fn.
void RadixSort(int N2)
{
    int Digit,i,RadixNo;
    char Txt[2];
    for(Digit=2;Digit>=0;Digit--) //Count down
    {
        printf("[Digit : %d]==>\n",3-Digit);
        for(i=0;i<N2;i++) //Data[] counter
        {
            itoa(Data[i],Txt,10); //convert Integer to Text
            [itoa(input,output,base)]
            RadixNo=Txt[Digit]-48; //convert Text that Digit to Number
            Push(RadixNo,Data[i]); //Pust Data to Stack at that Radix
        }
    }
}

```

```

    } //End for
    ReadStack();
    DispData(N2);
    ClearStackPT();
} //End for
} //End Fn.
int main()
{
    printf("ASCENDING RADIX SORT\n");
    printf("=====\n");
    N=16; //Change amount of N here
    N1=N; //Keep N to N1 Because N is 0 when finished RadixSort()!! I
don't understand
    PrepareRawData(N);
    printf("Raw Data...\n");
    DispData(N);
    printf("-----\n");
    printf("Processing Data...\n");
    RadixSort(N);
    //printf("N= %d\n",N); //Problem point!!
    printf("-----\n");
    printf("-Finished\n");
    printf("Sorted Data : \n");
    DispData(N1); //Sorted Data
    getch();
    return(0);
} //End Main

```

```

C:\Users\LENOVO\Downloads\Code\Code DataStructure65\25. Radix Sort\Radix Sort.exe
ASCENDING RADIX SORT
=====\n
Raw Data...\n
640 362 296 225 640 496 489 808 417 696 773 159 994 336 133 295
-----\n
Processing Data...\n
[Digit : 1]==>
640 640 362 773 133 994 225 295 296 496 696 336 417 808 489 159
[Digit : 2]==>
808 417 225 133 336 640 640 159 362 773 489 994 295 296 496 696
[Digit : 3]==>
133 159 225 295 296 336 362 417 489 496 640 640 696 773 808 994
-----\n
Sorted Data : \n
133 159 225 295 296 336 362 417 489 496 640 640 696 773 808 994

```

รูปที่ 4.13 แสดงผลการทำงานของโปรแกรมเรียงข้อมูลแบบเรดิก